26

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

인터셉터와 AOP

제2부 NestJS 핵심 · 6장 요청 파이프라인

모든 핸들러 앞뒤에 같은 일을 붙이는 관문



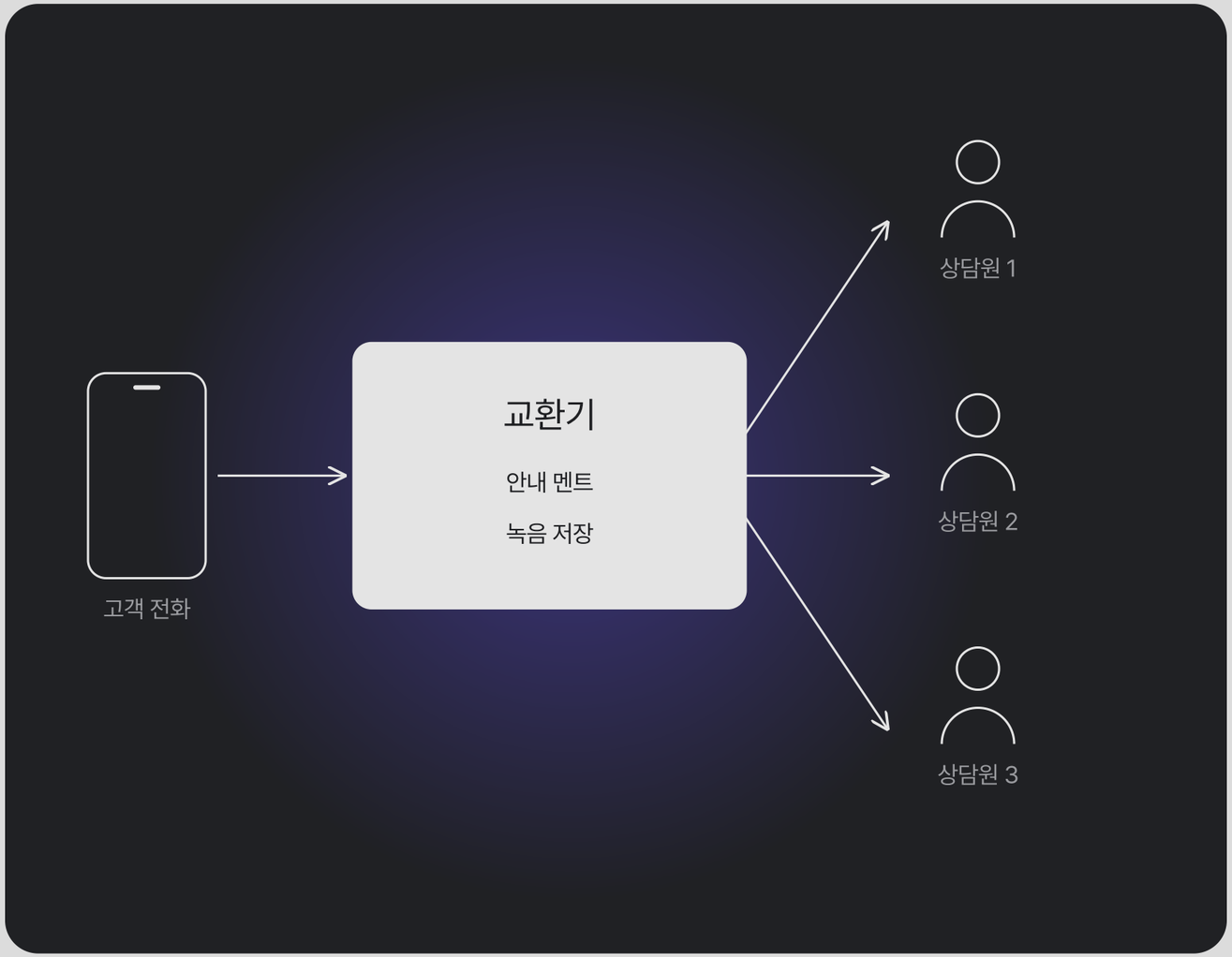
© 한상훈 NestJS 백엔드 마스터

녹음 안내 멘트

→ 어느 상담원에게 연결되든 같은 안내

→ 상담원이 매번 말하지 않음, 교환기가 앞에 붙임

→ 통화가 끝나면 녹음 파일 저장도 교환기가



© 한상훈 NestJS 백엔드 마스터

핸들러마다 붙인 측정 코드

seats.controller.ts

```
@Post(':seatId/hold')
hold(@Param('performanceId', ParseIntPipe) performanceId: number,
     @Param('seatId', SeatIdPipe) seatId: string) {
  const started = Date.now();
  const seat = this.seatsService.hold(performanceId, seatId);
  this.logger.log(`hold ${Date.now() - started}ms`);
  return seat;
}
```

→ 가정: 이 세 줄을 핸들러마다 복사

© 한상훈 NestJS 백엔드 마스터

붙여 넣기의 결과

- → 새 핸들러: 복사를 잊으면 측정에서 빠짐
- → 예외가 나면: 마지막 줄까지 가지 못해 기록 없음
- → 업무 코드 사이에 측정 코드가 섞임
- → 같은 종류의 일: 응답에 서버 버전 헤더, 요청마다 감사 기록
- → 로그 형식을 바꾸려면 핸들러 수십 곳 수정

© 한상훈 NestJS 백엔드 마스터

인터셉터

- → 핸들러 실행을 감싸는 클래스
- → `NestInterceptor` 구현, `intercept(context, next)` 하나
- → `context`: 실행 컨텍스트 (어느 클래스의 어느 핸들러인지)
- → `next.handle()`: 핸들러를 실행하고 결과를 Observable로 돌려줌
- → `next.handle()` 전: 앞부분, Observable 연산자: 뒷부분

© 한상훈 NestJS 백엔드 마스터

next.handle()과 Observable

→ Observable: 값이 나중에 흘러나오는 통로

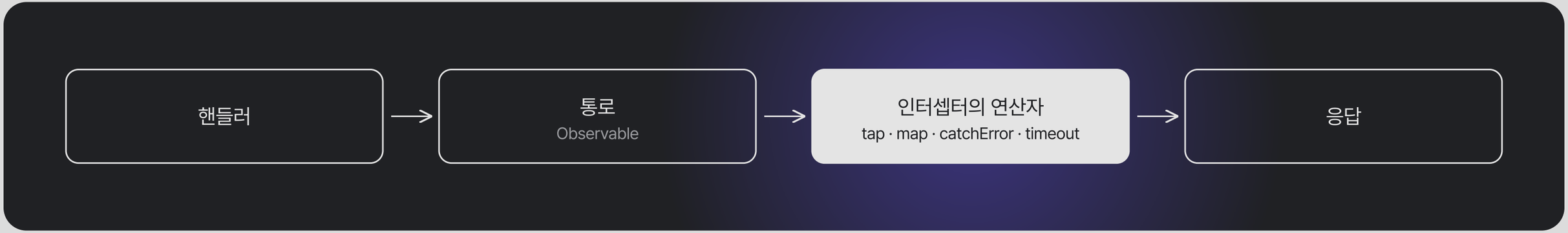
→ 여기서 흐르는 값: 핸들러가 돌려준 응답 하나

→ 예외: 값 대신 에러가 흐름

→ 인터셉터가 Observable을 돌려줌 → Nest가 구독해서 응답 전송

→ `next.handle()`을 부르지 않으면 핸들러 실행 없음

→ rxjs는 인터셉터 파일 안에만, 서비스·컨트롤러는 값이나 Promise 그대로



© 한상훈 NestJS 백엔드 마스터

tap과 map

연산자	하는 일	쓰는 곳
`tap`	흐르는 값을 보기만 함, 값은 그대로	로그, 측정
`map`	흐르는 값을 다른 값으로 바꿈	응답 감싸기, 필드 정리
`catchError`	흐르는 에러를 다른 에러나 값으로	에러 바꾸기
`timeout`	정한 시간 안에 값이 없으면 에러	시간 제한

데이터 블록: rxjs 7.8.x (package.json ^7.8.1, 2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

TimingInterceptor

timing.interceptor.ts

```ts
@Injectable()
export class TimingInterceptor implements NestInterceptor {
  private readonly logger = new Logger('Timing');

  intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
    const started = performance.now();
    const http = context.switchToHttp();
    const req = http.getRequest<Request>();
    const name = `${context.getClass().name}.${context.getHandler().name}`;
    const done = (result: string) => {
      const ms = (performance.now() - started).toFixed(1);
      this.logger.log(`${req.headers['x-request-id']} ${name} ${result} ${ms}ms`);
    };
    return next.handle().pipe(
      tap({
        next: () => done(String(http.getResponse<Response>().statusCode)),
        error: (err) => done(`실패(${err.constructor.name})`),
      }),
    );
  }
}
```

© 한상훈 NestJS 백엔드 마스터

APP_INTERCEPTOR

app.module.ts

```ts
@Module({
  // imports, controllers 그대로
  providers: [AppService, { provide: APP_INTERCEPTOR, useClass: TimingInterceptor }],
})
export class AppModule {}
```

→ 모든 핸들러에 적용, 생성자 주입 가능

→ 핸들러 코드는 한 줄도 바뀌지 않음

© 한상훈 NestJS 백엔드 마스터

Timing 로그

서버 로그

```
[Timing] 75d55345-… SeatsController.findAll 200 0.4ms
[HTTP]   75d55345-… GET /performances/1/seats?row=3 200 1.2ms
[Timing] 8dbf3dd9-… SeatsController.hold 201 0.2ms
[HTTP]   8dbf3dd9-… POST /performances/1/seats/A-1-1/hold 201 0.8ms
[Timing] 9e0560f2-… SeatsController.hold 실패(ConflictException) 0.4ms
[HTTP]   9e0560f2-… POST /performances/1/seats/A-1-1/hold 409 0.9ms
[Timing] 19a9809f-… ReservationsController.reserve 201 0.8ms
[HTTP]   19a9809f-… POST /reservations 201 1.7ms
```

→ 같은 요청 ID로 두 줄: 핸들러 시간과 요청 전체 시간

→ 가정: Timing 1ms, HTTP 300ms → 핸들러 밖(가드, 압축·전송)부터 확인

데이터 블록: 강사 PC 실측 0.2~1.7ms (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

실패도 남기기

- → `tap(fn)`: 값이 흐를 때만 실행 → 예외로 끝난 요청은 기록 없음
- → `tap({ next, error })`: 에러가 흐를 때도 실행
- → `performance.now()`: 앞으로만 가는 시계, `Date.now()`는 시스템 시계 조정 때 튈 수 있음
- → 측정값 수집 서비스를 생성자로 받아 보내기도 가능
- → 에러를 붙잡지 않음: 그대로 예외 필터로

© 한상훈 NestJS 백엔드 마스터

AOP

- → 관점 지향 프로그래밍(Aspect-Oriented Programming)
- → 횡단 관심사(cross-cutting concern): 여러 기능에 똑같이 걸치는 일
- → 예: 로그, 측정, 트랜잭션 경계, 캐시, 권한
- → 업무 코드에는 업무만, 걸치는 일은 한 곳에서 감싸기
- → 예: 예매 서비스 안의 트랜잭션 시작·커밋 → 감싸는 쪽으로 빼면 예매 규칙만 남음
- → Nest에서 감싸는 도구: 인터셉터(그리고 가드, 파이프, 예외 필터)

© 한상훈 NestJS 백엔드 마스터

두 줄이 남는 요청, 한 줄만 남는 요청

○ ○ ○ 서버 로그

```
[HTTP]   91a42185-… POST /box-office/performances/1/seats/A-1-2/sell 401 15.4ms

[Timing] 442ac26e-… SeatsController.findAll 실패(BadRequestException) 0.7ms
[HTTP]   442ac26e-… GET /performances/1/seats?row=abc 400 1.1ms
```

→ 가드가 막은 401: Timing 없음

→ 파이프가 막은 400: Timing 있음 (인터셉터가 파이프를 감쌈)

© 한상훈 NestJS 백엔드 마스터

요청 로그와 Timing 로그

	요청 로그 (미들웨어)	Timing (인터셉터)
보는 요청	전부 (없는 주소, 401, 본문 초과 포함)	가드를 통과한 요청
아는 것	메서드, 경로, 상태 코드	컨트롤러와 핸들러 이름
재는 범위	들어와서 응답이 다 나갈 때까지	파이프와 핸들러
쓰는 곳	에러율, 접근 기록	느린 핸들러 찾기

→ 인터셉터로 빼지 않을 것: 좌석이 남았는지 같은 업무 규칙 (서비스에)

© 한상훈 NestJS 백엔드 마스터

{ data }로 감싸기

```ts
@Injectable()
export class WrapInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
    return next.handle().pipe(map((data) => ({ data })));
  }
}
```

curl

```
$ curl "localhost:4026/performances/1/seats?row=1&status=HELD"
{"data":[{"id":"A-1-1","label":"A구역 1열 1번","status":"HELD"}]}
$ curl "localhost:4026/performances/1/seats?row=abc"
{"type":"about:blank","title":"Bad Request","status":400,"errors":[…], …}
```

→ 성공만 감싸짐, 400은 그대로

→ map의 다른 쓰임: 나가는 응답에서 내부용 필드 지우기

© 한상훈 NestJS 백엔드 마스터

핸들러를 부르지 않는 인터셉터

```
intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
  const key = context.switchToHttp().getRequest<Request>().originalUrl;
  const hit = this.cache.get(key);
  if (hit && Date.now() - hit.at < 3000) {
    this.logger.log(`캐시 응답 ${key}`);
    return of(hit.body);              // next.handle()을 부르지 않음
  }
  return next.handle().pipe(tap((body) => this.cache.set(key, { at: Date.now(), body })));
}
```

SeatMapCacheInterceptor (시연용, 프로젝트에 남기지 않음)

© 한상훈 NestJS 백엔드 마스터

3초 동안 같은 답

요청 순서

```
POST …/seats/A-1-1/hold
GET  …/seats?row=1&status=HELD  → A-1-1 (좌석 목록 핸들러 실행)
POST …/seats/A-1-2/hold
GET  …/seats?row=1&status=HELD  → A-1-1 만  (캐시 응답, 핸들러 실행 없음)
(3초 뒤)
GET  …/seats?row=1&status=HELD  → A-1-1, A-1-2 (좌석 목록 핸들러 실행)
```

→ 기억해 둔 응답 = 데이터의 사본, 원본이 바뀌어도 사본은 그대로

© 한상훈 NestJS 백엔드 마스터

TimeoutInterceptor

timeout.interceptor.ts

```ts
// 2초 안에 끝나지 않으면 504
@Injectable()
export class TimeoutInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
    return next.handle().pipe(
      timeout(2000),
      catchError((err) =>
        throwError(() => (err instanceof TimeoutError ? new GatewayTimeoutException('처리 시간이 2초를 넘었습니다') : err)),
      ),
    );
  }
}
```

→ ReservationsController에 `@UseInterceptors(TimeoutInterceptor)`

© 한상훈 NestJS 백엔드 마스터

504 뒤의 결제 완료

curl · 서버 로그

```
$ FAKE_PG_DELAY_MS=5000 (가짜 결제사가 5초 걸림)
06:11:57.389 요청 보냄
{"type":"about:blank","title":"Gateway Timeout","status":504,
 "detail":"처리 시간이 2초를 넘었습니다", …} [504] 2.009693s
06:11:59.485 응답 받음

[Timing] 8a6f6579-… ReservationsController.reserve 실패(GatewayTimeoutException) 2006.7ms
[HTTP]   8a6f6579-… POST /reservations 504 2008.4ms
06:12:02.455 [예매] A-1-1 결제 완료
```

시각: UTC, 시연용으로 예매 로그에 시각 추가

데이터 블록: 강사 PC 실측: 504 2.009693s, 결제 완료 요청 뒤 약 5.07초, FAKE_PG_DELAY_MS=5000 (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

끊긴 건 응답

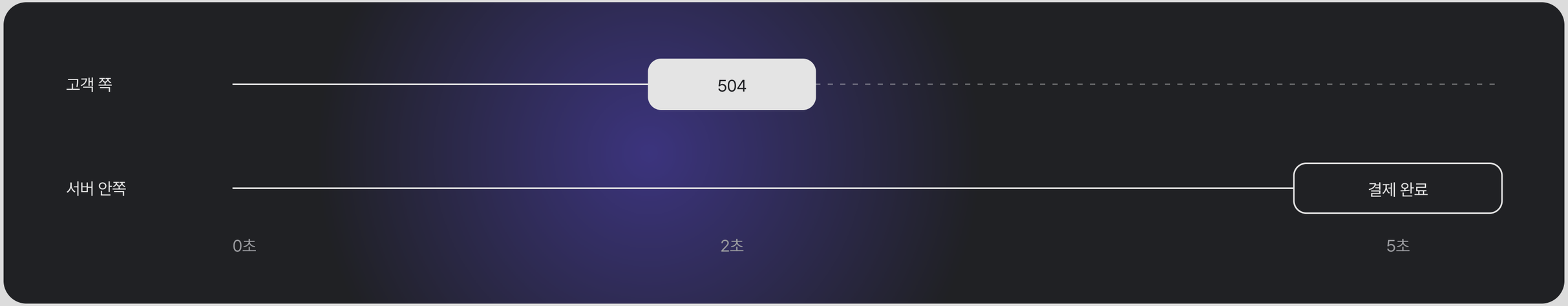
→ `timeout`: Observable 구독을 끊고 에러를 흘려보냄

→ 핸들러가 돌려준 Promise: 이미 시작, 끊을 방법 없음

→ 결제사 호출, 예매 저장, 확정 문자: 계속 진행

→ 멈추려면: 호출하는 쪽이 취소 신호(`AbortSignal`)를 받아 결제사 요청을 끊어야 함

→ 고객: 화면은 시간 초과, 몇 초 뒤 예매 확정 문자



© 한상훈 NestJS 백엔드 마스터

고객이 다시 누르면

→ 고객 화면: 2초에 "처리 시간 초과"

→ 실제: 5초에 결제 승인, 예매 저장

→ 고객이 다시 결제 → 두 번째 결제 (11만 원 좌석 → 카드 문자 11만 원 두 번)

→ 504: 앞단이 기다리다 포기했다는 뜻, 일시적 실패로 보고 자동 재시도하기 쉬움

→ 결과를 모르는 결제: 실패라고 알린 요청이 성공해 있는 경우

© 한상훈 NestJS 백엔드 마스터

인터셉터가 할 수 있는 것과 없는 것

할 수 있음	할 수 없음
핸들러 앞뒤에 코드 실행 (측정, 로그)	가드가 막은 요청 보기
응답 값 바꾸기 (`map`)	예외 응답의 본문 만들기 (예외 필터 몫)
에러를 다른 에러로 바꾸기 (`catchError`)	이미 시작한 핸들러의 일 멈추기
핸들러 건너뛰기 (`next.handle()`을 부르지 않음)	파이프 전에 검증된 값 보기

→ 여러 개: 바깥(전역 → 컨트롤러 → 메서드)부터 감싸고 안쪽부터 풀림

→ 프로젝트에 남긴 것: TimingInterceptor (`APP_INTERCEPTOR`)

© 한상훈 NestJS 백엔드 마스터

인터셉터

"인터셉터는 핸들러 앞뒤를 감싸서, 모든 핸들러에 똑같이 붙는 일을 한 곳에 둡니다."